

SRG093X 系列 M33 开发指导

短距离产品

版本：1.0.0

日期：2026-09-17

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市松江区泗泾镇外婆泾路 8 号 邮编：201601

电话：+86 21 5108 6236

邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<https://www.quectel.com.cn/contact>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：

<https://www.quectel.com.cn/contact?tab=t> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持您的产品设计。您须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。您知悉并同意，移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档，增加、修改或重述后的文档对您具有约束力。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，任何形式的购买都不可被视为授予除正常的非独家、免版税的产品使用许可之外的任何许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法、侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何明示、暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2026，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2026.

文档历史

修订记录

版本	日期	作者	变更表述
-	2026-08-13	Neil Shi	文档创建
1.0.0	2026-09-17	Billy Zhang	临时版本

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
1.1. 硬件环境	7
2 启动 M33 核.....	8
2.1. M33 核硬件介绍.....	8
2.2. M33 核调试串口输出	9
2.3. 启动 M33 核	9
2.3.1. U-Boot 阶段启动 M33 核	9
2.3.2. 内核阶段启动 M33 核	11
3 M33 核的 FreeRTOS 开发	13
3.1. MCU SDK 目录	13
3.2. 在 Yocto 中编译 M33 镜像	15
4 M33 核的 Zephyr 开发.....	19
4.1. 配置 VS Code 的 Zephyr 编译环境.....	19
4.1.1. 安装 MCUXpresso for VS Code 插件.....	19
4.1.2. 安装 MCUXpresso Installer	20
4.1.3. 安装 Zephyr 工具链和安装程序	21
4.1.4. 安装 Zephyr 代码仓库.....	21
4.1.5. 安装 MCUXpresso 工具.....	22
4.1.6. 安装问题及解决方法.....	23
4.2. 在 VS Code 中编译 M33 镜像.....	25
4.2.1. 配置 Zephyr 工程.....	25
4.2.2. 编译 M33 镜像	28
4.3. 在 Ubuntu 中编译 M33 镜像.....	29
4.4. 集成 M33 镜像	30
4.4.1. 复制 M33 镜像至 Boot 分区	30
4.4.2. 打包 M33 镜像到 Rootfs 分区.....	30
4.5. 启动验证	31
5 A55 核的 Zephyr 开发	33
6 附录 参考文档及术语缩写	34

表格索引

表 1: 硬件环境.....	7
表 2: 硬件信息.....	8
图 3: 串口配置.....	9
表 4: MCU SDK 目录.....	15
表 5: DTS 文件.....	26
表 6: 内核配置文件.....	26
表 7: 参考文档.....	34
表 8: 术语缩写.....	34

图片索引

图 1: 硬件信息概览.....	8
图 2: 选择目标硬件与工具链.....	13
图 3: 构建 SDK.....	14
图 4: 下载 SDK 源码包.....	14
图 5: MCUXpresso for VS Code 插件.....	20
图 6: MCUXpresso Installer 下载页面.....	20
图 7: MCUXpresso Installer 界面.....	21
图 8: 导入 SDK.....	22
图 9: 安装 MCUpresso 工具.....	22
图 10: 打开 sysdm.cpl.....	24
图 11: 添加变量.....	24
图 12: 导入 hello_world 例程代码.....	25
图 13: 导入成功后的界面.....	26
图 14: 配置内核.....	27
图 15: 编译镜像.....	28
图 16: 编译成功.....	28
图 17: 编译生成的文件.....	29

1 引言

移远通信 SRG093X 系列模块集成了一颗主频为 250 MHz 的 Arm® Cortex®-M33 核处理器，用于实时计算和控制相关功能。本文档介绍如何在移远通信 SRG093X 系列模块上启动 M33 核并进行应用开发。

1.1. 硬件环境

表 1：硬件环境

硬件名称	数量
移远通信 SRG093X-TE-A / SRG093X-W-TE-A	1
移远通信 SR-IMXM-EVB	1
USB Type-C 线	1
电源线	1

2 启动 M33 核

2.1. M33 核硬件介绍

Arm® Cortex®-M33 核基于 Armv8 M 架构，是一款兼具低功耗、安全性和实时性能的 32 位 MCU 核。

表 2：硬件信息

硬件	说明
主频	250 MHz
缓存配置	16 KB 代码缓存 + 16 KB 系统缓存
硬件组件	FPU、MPU、NVIC
存储系统	256 KB TCM/OCRAM w/ECC

硬件信息如下图所示：

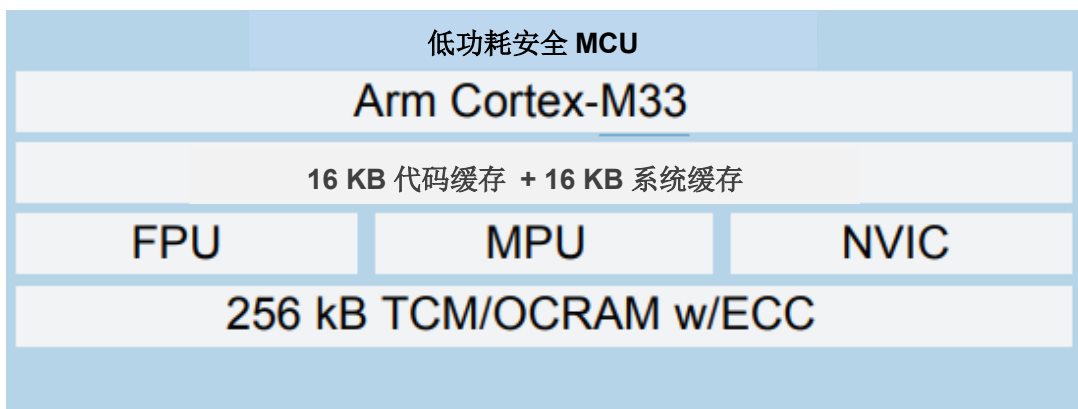


图 1：硬件信息概览

2.2. M33 核调试串口输出

将 SR-IMXM-EVB 的 DEBUG_UART USB 口连接至 PC 的 USB 口之后，系统将会枚举出 4 个串口；其中一路为 M33 核专属 UART 口。用户可打开全部 4 路串口，在完成 M33 核启动流程之后，该专属 UART 口便会打印对应的运行日志。

图 3：串口配置

参数	配置
波特率	115200
数据位	8
校验位	无
停止位	1
流控	无

2.3. 启动 M33 核

Linux 编译工程默认内置若干简易 FreeRTOS 示例文件，使用这些文件按照以下步骤启动 M33 核。更多详细内容，请参考 <https://www.nxp.com.cn/docs/en/user-guide/UG10163.pdf> 的第 4.7.4 章。

M33 核支持两种相互独立的启动方式：在 U-Boot 阶段启动和在内核阶段启动。用户可以根据实际需求任选一种。

2.3.1. U-Boot 阶段启动 M33 核

当出现下述日志并开启 3 秒倒计时，按下任意按键即可进入 U-Boot 模式。

```
Hit any key to stop autoboot: 0
```

执行如下命令在 U-Boot 阶段启动 M33 核。

步骤一： 拷贝 M33 核镜像（以 *power_rode_Switch_rtos_imx93. Bin* 为例）至 DDR（起始地址 0x80000000）。

```
u-boot => fatload mmc 0:1 80000000 mcore-demos/imx93-11x11-evk_m33_TCM_power_mode_switch.bin
```

步骤二： 将镜像从 DDR 拷贝至 A-core TCM 对应的存储地址（0x201e0000）。

```
u-boot => cp.b 0x80000000 0x201e0000 0x10000
```

步骤三： 从 TCM 起始地址（0x1ffe0000）启动 M33 核。

```
u-boot => bootaux 0x1ffe0000 0
```

成功启动 M33 核之后，M33 调试串口将会输出如下打印信息：

```
##### Power Mode Switch Task #####

Build Time: Apr 30 2024--04:18:07
Core Clock: 200000000Hz

Select the desired operation
Press A to enter: Normal RUN mode
Press B to enter: WAIT mode
Press C to enter: STOP mode
Press D to enter: SUSPEND mode
Press W to wakeup A55 core
Press M for switch M33 Root Clock frequency between OD/ND.

Waiting for power mode select..
```

M33 核扩展内存相关说明如下：

1) 内存介绍

内存区域	核	起始地址	地址区间（容量）
DDR	A55/M33 核	0x80000000	2 GB: 0x80000000~0xFFFFFFFF
			1 GB: 0x80000000~0xBFFFFFFF
TCM（代码）	A55 核	0x201e0000	128 KB: 0x201E0000~0x201FFFFF
TCM（代码）	M33 核	0x1FFE0000	128 KB: 0x1FFE0000~0x1FFFFFFF

2) Fatload 参数介绍

通过 **fatload** 命令从 eMMC 的 Boot 分区加载 FAT 格式文件 *power_mode_switch_rtos_imx93.bin*，并将该镜像拷贝至内存起始地址 0x80000000。

MMC 分区（用于 eMMC、SD 卡加载）：

- MMC 0:1 eMMC 的 Boot 启动分区（存放内核镜像、DTB 等）
- MMC 0:2 eMMC 的根文件系统分区
- MMC 1:1 SD 卡的 Boot 启动分区（存放内核镜像、DTB 等）
- MMC 1:2 SD 卡的根文件系统分区

3) cp.b 命令

该命令用于将位于 DDR 地址 0x80000000 处的固镜像拷贝至 A-coreTCM 地址 0x201e0000，传输大小为 0x10000 字节。

命令格式：

```
cp.b <Starting address of DDR in core-A> <Starting address of TCM in core-A> <size>
```

4) bootaux 命令

该命令用于在 A55 核的 U-Boot 环境中将 M33 核 TCM 的起始地址写入 SRC-GPR19 寄存器，以此启动 M33 核。

命令格式：

```
bootaux <Starting address of TCM in core-M> 0
```

2.3.2. 内核阶段启动 M33 核

本章介绍如何借助 A55 核的 RPROC 功能启动 M33 核。

A55 核启动后，在 A55 的调试串口中执行如下命令以启动 M33 核。

步骤一： 加载 imx_rpmsg_tty 驱动。

```
modprobe imx_rpmsg_tty
```

步骤二： 将 M33 镜像名称写入 RPROC 模块。

```
echo imx93-11x11-evk_m33_TCM_rpmsg_lite_str_echo_rtos.elf >
/sys/class/remoteproc/remoteproc0/firmware
```

步骤三： 通过 RPROC 启动 M33 核。

```
echo start > /sys/class/remoteproc/remoteproc0/state
```

A55 核调试串口的输出日志如下：

```
[ 448.338304] remoteproc remoteproc0: powering up imx-rproc
[ 448.342169] remoteproc remoteproc0: Booting fw image rpmsg_lite_str_echo_rtos_remote_cm33.elf, size 471508
[ 448.866121] rproc-virtio rproc-virtio.3.auto: assigned reserved memory node vdevbuffer@a4020000
[ 448.866882] virtio_rpmsg_bus virtio0: rpmsg host is online
[ 448.866922] rproc-virtio rproc-virtio.3.auto: registered virtio0 (type 7)
[ 448.866942] rproc-virtio rproc-virtio.4.auto: assigned reserved memory node vdevbuffer@a4020000
[ 448.867438] virtio_rpmsg_bus virtio1: rpmsg host is online
[ 448.867469] rproc-virtio rproc-virtio.4.auto: registered virtio1 (type 7)
[ 448.867475] remoteproc remoteproc0: remote processor imx-rproc is now up
[ 450.118249] virtio_rpmsg_bus virtio1: creating channel rpmsg-virtual-tty-channel addr 0x1e
[ 450.118519] imx_rpmsg_tty virtio1.rpmsg-virtual-tty-channel.-1.30: new channel: 0x400 -> 0x1e!
[ 450.118900] Install rpmsg tty driver!
[ 450.128722] rpmsg tty_cb68 65 6c 6c 6f 20 77 6f 72 6c 64 21      hello world!
```

同时，M33 核调试串口输出日志如下：

```
RPMSG String Echo FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
Reply send ok
```

步骤四： 测试 RPMSG 通道；A55 核经由 ttyRPMSG30 向 M33 核发送消息。

```
echo "test123 " > /dev/ttyRPMSG30
```

步骤五： M33 核接收到 A55 核下发的数据以后，将相同数据回传给 A55 端；查看 A55 端 demsg 的打印信息。

```
[ 450.118900] Install rpmsg tty driver!
[ 450.128722] rpmsg tty_cb68 65 6c 6c 6f 20 77 6f 72 6c 64 21      hello world!
[ 877.419362] rpmsg tty_cb74 65 73 74 31 32 33      test123
[ 877.423764] rpmsg tty_cb0d 0a      ..
```

步骤六： 通过 RPROC 停止 M33 核。

```
echo stop > /sys/class/remoteproc/remoteproc0/state
```

3 M33 核的 FreeRTOS 开发

本章介绍如何使用第 2.3 章所述 Linux 镜像中集成的 .bin 和 .elf 格式 FreeRTOS 可执行文件，进行 M33 核的 FreeRTOS 开发。

3.1. MCU SDK 目录

用户可从 <https://mcuxpresso.nxp.com/select> 下载 MCU 演示程序源码，具体操作步骤如下：

步骤一： 在“MCUXpresso SDK Builder”界面的搜索框中输入“imx93”进行搜索。在搜索结果中的“Boards”列表中选择“**MCIMX93-EVK (MIMX9352xxxxM)**”。也可以在“Processors”列表中选择“**MIMX9332xxxxM**”。确认选择后，单击“**ARMGCC**”按钮。

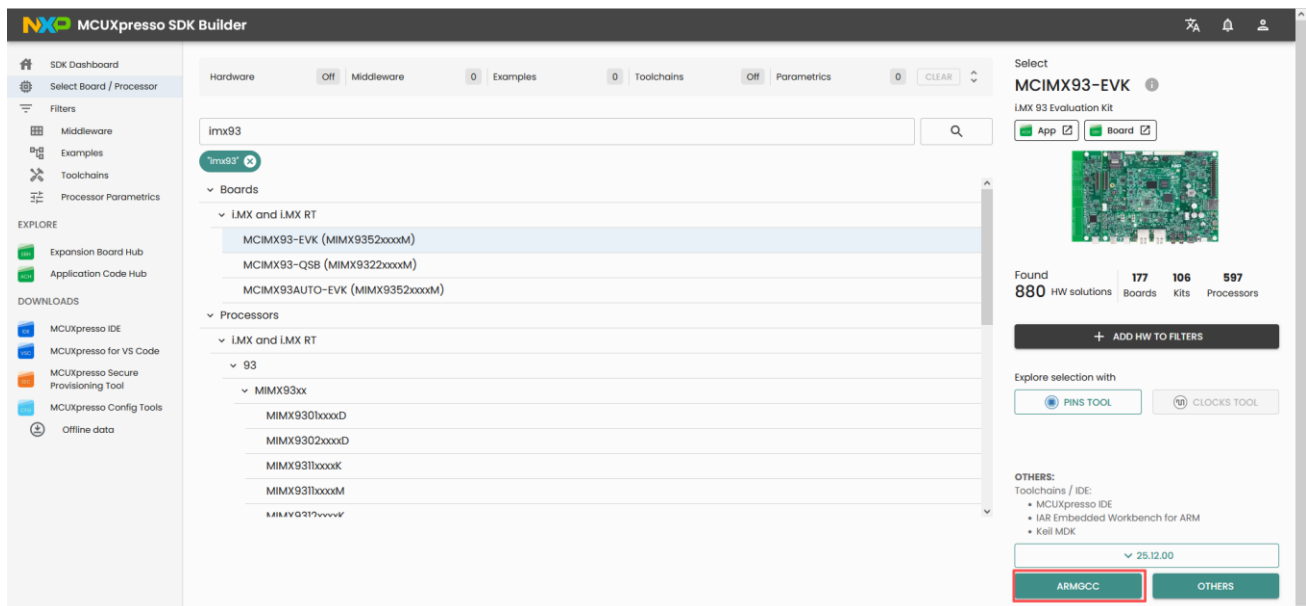


图 2：选择目标硬件与工具链

步骤二： 打开“Developer Environment Settings”页面，将 Host OS 选择为“**Linux**”，并将 Toolchain / IDE 选择为“**ARM GCC / MCUXpresso for VS Code**”。按照下图所示，确保列表中已勾选所需的中间件组件（如 FreeRTOS、Multicore、lwIP 等）。单击界面右下角的“**BUILD SDK**”按钮。

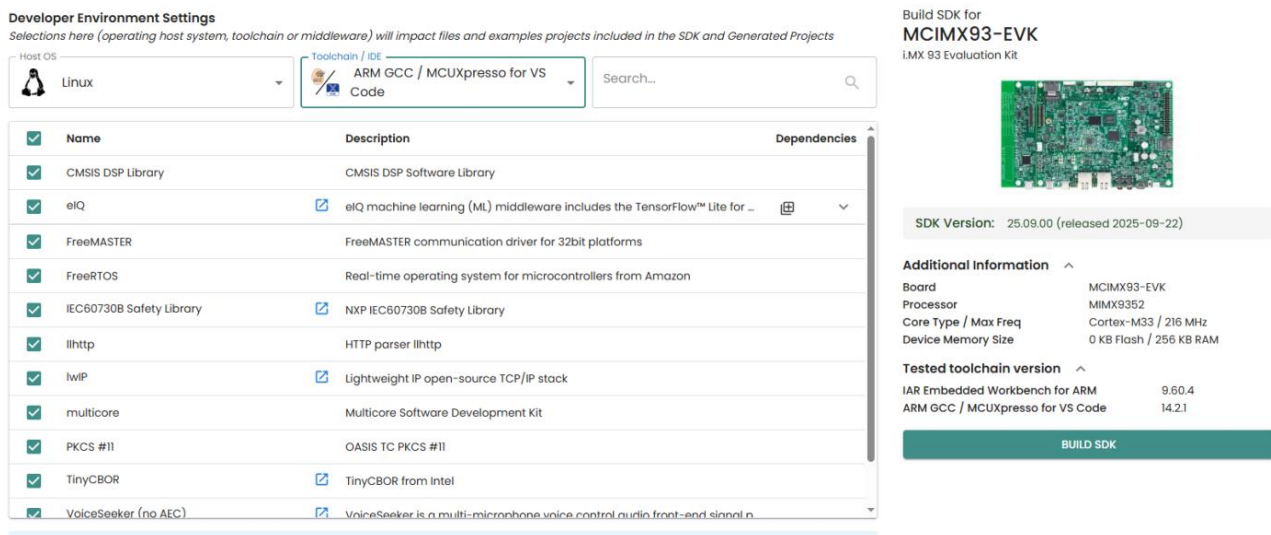


图 3：构建 SDK

步骤三： SDK 构建完成后，在“MCUXpresso SDK Dashboard”界面中找到已构建的 SDK 条目（例如 SDK_25.09.00_MCIMX93-EVK）。单击该条目旁边的下载图标，下载 SDK 源码包。

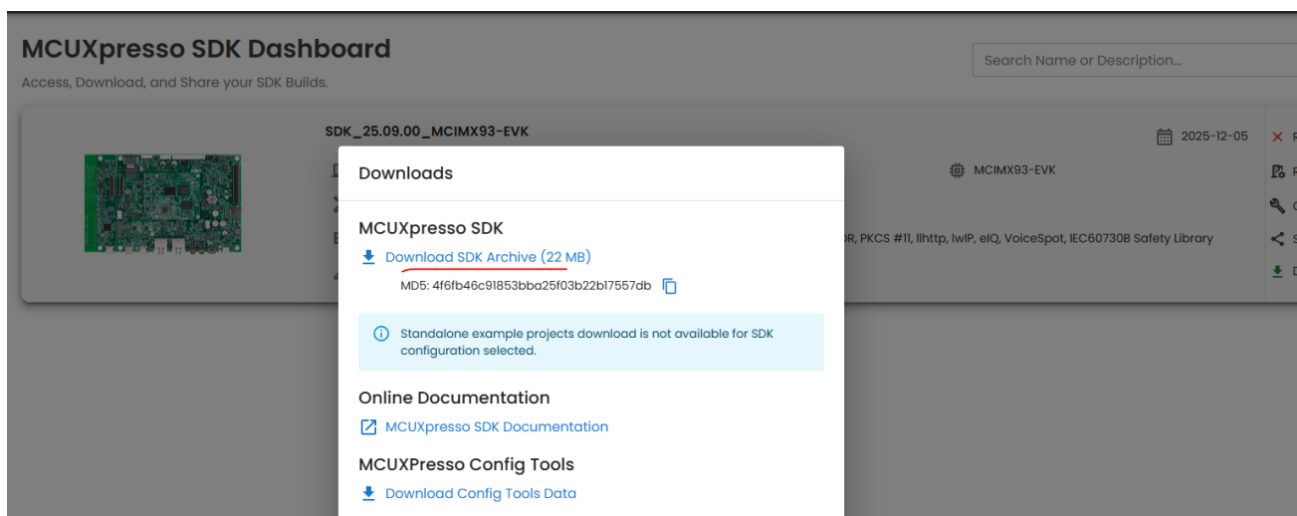


图 4：下载 SDK 源码包

SRG093X 系列模块源码中不包含 SDK 代码，因此需按照上述步骤手动下载。本文件基于 <SRG09x_source_code>/yocto/sources/quectel/mcu_sdk/目录中的源码介绍 M33 的 FreeRTOS 开发流程。

有关使用 CMake 构建 FreeRTOS 可执行文件的说明，请参阅 [mcuxsdk/README.md](#)。每个演示程序/示例均提供了相应的构建说明。以下链接 i.MX 93 的“Hello World”演示程序为例介绍具体用法：
https://mcuxpresso.nxp.com/mcuxsdk/latest/html/examples/boards/mcimx93evk/demo_apps/hello_world/example_board_readme.html。

表 4: MCU SDK 目录

目录	说明
<i>boards</i>	存放板级支持包（包括示例代码：裸机运行例程和 FreeRTOS 镜像运行例程）。
<i>devices</i>	存放芯片级代码（寄存器头文件、启动代码和驱动程序）。
<i>rtos</i>	存放 FreeRTOS 内核源代码。
<i>middleware</i>	存放 Audio-voice、llhttp、Multicore、pkcs11、tinycbor、eiq 等中间件组件。
<i>components</i>	存放硬件驱动 API，包括 Codec、SPI、I2C、传感器和 PMIC 等接口。

备注

本文档使用的 SDK 版本为 SDK_25.09.00。若在使用其他版本的 SDK 时遇到任何问题，请联系移远通信技术支持。

3.2. 在 Yocto 中编译 M33 镜像

本章介绍如何通过 Yocto 集成方式编译新的 M33 镜像。以下示例基于如下目录中的代码：
`<SRG09x_source_code>/sources/quectel/mcu_sdk/boards/mcimx93evk/multicore_examples/rpmsg_lite_str_echo_rtos`

按照 Yocto 编译规则，将 *rpmsg_lite_str_echo_rtos* 的源代码编译为*.bin 和 *.elf 镜像文件。编译完成后，M33 固件将被打包到 SRG093X 系列模块镜像包中。其中，*.bin 文件存放在 boot 镜像分区的 *mcore-demos* 目录中，*.elf 文件存放在 rootfs 分区的 */usr/lib/firmware* 目录中。

修改 M33 固件配置如下：

```
diff --git a/yocto/sources/meta-imx/meta-imx-bsp/conf/layer.conf b/yocto/sources/meta-imx/meta-imx-bsp/conf/layer.conf
index e594969c..488e57bc 100644
--- a/yocto/sources/meta-imx/meta-imx-bsp/conf/layer.conf
+++ b/yocto/sources/meta-imx/meta-imx-bsp/conf/layer.conf
@@ -425,6 +425,7 @@ @@ IMAGE_BOOT_FILES:append:imx93-11x11-lpddr4x-quec = "\
     mcore-demos/imx93-11x11-evk_m33_TCM_rpmsg_lite_pingpong_rtos_linux_remote.bin \
     mcore-demos/imx93-11x11-evk_m33_TCM_rpmsg_lite_str_echo_rtos.bin \
     mcore-demos/imx93-11x11-evk_m33_TCM_sai_low_power_audio.bin \
+     mcore-demos/rpmsg_lite_str_echo_rtos_remote.bin \
"
KERNEL_DEVICETREE:remove:imx93-11x11-lpddr4x-quec = "\
```

```

freescale/imx93-11x11-quec-lpspi-slave.dtb \
diff --git a/yocto/sources/meta-imx/meta-imx-bsp/recipes-quectel/quec-mcore/quec-mcore-
rpmsgtest.bb b/yocto/sources/meta-imx/meta-imx-bsp/recipes-quectel/quec-mcore/quec
-mcore-rpmsgtest.bb
new file mode 100644
index 00000000..f5334323
--- /dev/null
+++ b/yocto/sources/meta-imx/meta-imx-bsp/recipes-quectel/quec-mcore/quec-mcore-
rpmsgtest.bb
@@ -0,0 +1,52 @@
+SUMMARY = "i.MX93 M33 RPMSG Lite String Echo Firmware"
+DESCRIPTION = "rpmsg_lite_str_echo_rtos example for i.MX93 M33 core"
+#LICENSE = "CLOSED"
+#LIC_FILES_CHKSUM = ""
+LICENSE = "Proprietary"
+LIC_FILES_CHKSUM = "file://COPYING;md5=abdd0805e91a9d0af5553d03a8cfa726"
+
+PR = "r0"
+
+DEPENDS = "cmake-native ninja-native gcc-arm-none-eabi-native"
+inherit cmake deploy
+
+FILESPATH += "${BSPDIR}/sources/quectel:"
+SRC_URI = "file://mcu_sdk"
+S = "${WORKDIR}/mcu_sdk"
+
+do_configure[noexec] = "1"
+
+do_compile() {
+    cd ${S}
+    cd boards/mcimx93evk/multicore_examples/rpmsg_lite_str_echo_rtos/remote/armgcc
+    export ARMGCC_DIR=${STAGING_BINDIR_NATIVE}/..
+    ./build_all.sh
+}
+
+do_install() {
+    install -d ${D}${nonarch_base_libdir}/firmware
+    install -m 0644
+${S}/boards/mcimx93evk/multicore_examples/rpmsg_lite_str_echo_rtos/remote/armgcc/debug/*.el
f ${D}${nonarch_base_libdir}/firmware
+}
+
+DEPLOY_FILE_EXT      ?= "bin"
+DEPLOY_FILE_EXT:mx7ulp-nxp-bsp = "img"

```

```

+
+do_deploy () {
+    # Install the demo binaries
+    install -d ${DEPLOYDIR}/mcore-demos
+    install -m 0644
+${S}/boards/mcimx93evk/multicore_examples/rpmsg_lite_str_echo_rtos/remote/armgcc/debug/*.${
+DEPLOY_FILE_EXT} ${DEPLOYDIR}/mcore-demos/
+}
+
+addtask deploy after do_install
+
+FILES:${PN} += " \
+    /usr/ \
+    /usr/lib/ \
+    /usr/lib/firmware/ \
+    /usr/lib/firmware/quect_rpmsg_lite_str_echo_rtos_remote_cm33.elf \
+"
+
+INSANE_SKIP:${PN} = "arch ldflags textrel"
+
+FIRMWARE_PACKAGES += "${PN}"
+PACKAGE_ARCH = "${MACHINE_ARCH}"
diff --git a/yocto/sources/meta-imx/meta-imx-sdk/recipes-fsl/images/imx-image-multimedia.bb
b/yocto/sources/meta-imx/meta-imx-sdk/recipes-fsl/images/imx-image-multimedia.bb
index 2cccc175..9486b126 100644
--- a/yocto/sources/meta-imx/meta-imx-sdk/recipes-fsl/images/imx-image-multimedia.bb
+++ b/yocto/sources/meta-imx/meta-imx-sdk/recipes-fsl/images/imx-image-multimedia.bb
@@ -30,6 +30,7 @@ @@ IMAGE_FEATURES += " \
        bb.utils.contains('DISTRO_FEATURES',      'x11', 'x11-base x11-sato', \
                                                    ", d), d)} \
"
+IMAGE_INSTALL += " quec-mcore-rpmsgtest"

V2X_PKGS = ""
V2X_PKGS:mx8dxl-nxp-bsp = "packagegroup-imx-v2x"
diff --git a/yocto/sources/quectel/mcu_sdk/COPYING b/yocto/sources/quectel/mcu_sdk/COPYING
new file mode 100644
index 00000000..d1c06542
--- /dev/null
+++ b/yocto/sources/quectel/mcu_sdk/COPYING
@@ -0,0 +1,3 @@
+# Proprietary License
+Copyright (C) 2025 Quectel. All rights reserved.

```

+This software is proprietary to Quectel and may not be reproduced, distributed, or modified without permission.

将上述配置集成到 SRG093X 系列模块的源码中后，执行如下命令编译完整镜像，其中包含已编译的 M33 固件。生成的镜像文件路径为 `<SRG09x_source_code>/yocto/build-imx93quec/tmp/deploy/images/imx93quec/*wic.zst`。

```
MACHINE=imx93quec DISTRO=fsl-imx-xwayland source imx-setup-release.sh -b build-imx93quec  
bitbake imx-image-multimedia
```

4 M33 核的 Zephyr 开发

本章介绍如何快速搭建并运行 Zephyr 开发环境。首次搭建环境时，请参考 Zephyr 官方入门指南：https://docs.zephyrproject.org/latest/develop/getting_started/index.html。该指南提供了 Ubuntu、macOS 和 Windows 平台的操作说明，请根据实际开发环境进行选择。

第 4.1 章和第 4.2 章介绍了如何在 Windows 平台的 VS Code 中搭建编译环境。如果不使用 Windows 进行开发，请跳过这两个章节的操作。如需查看可视化操作流程，请参考官方的 MCUXpresso for VS Code 环境搭建视频：<https://www.nxp.com/design/design-center/training/TIP-GETTING-STARTED-WITH-MCUXPRESSO-FOR-VS-CODE>。

4.1. 配置 VS Code 的 Zephyr 编译环境

4.1.1. 安装 MCUXpresso for VS Code 插件

本节介绍如何在 Visual Studio Code 中安装 MCUXpresso for VS Code 插件，以支持嵌入式开发。安装步骤如下。

- 步骤一：单击左侧边栏中的插件“”。
- 步骤二：在界面顶部的搜索框中输入“mcuxpresso”。
- 步骤三：找到“MCUXpresso for VS Code”，在详情页面单击“Install”按钮，然后等待安装完成。

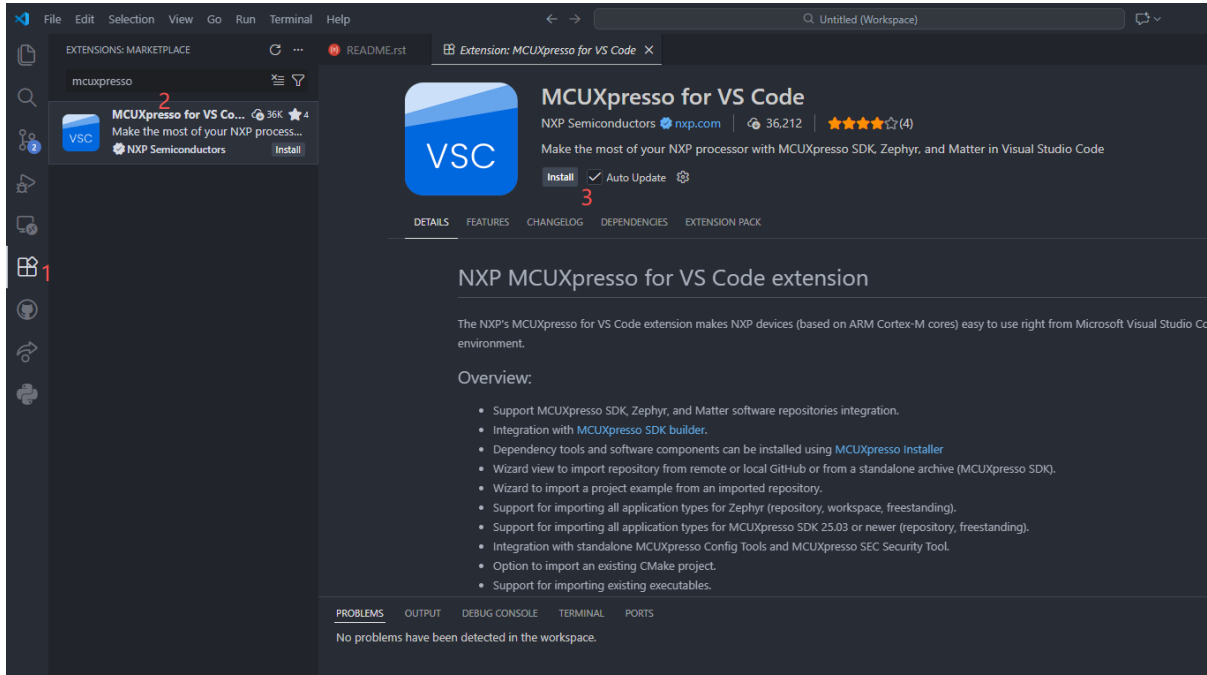


图 5: MCUXpresso for VS Code 插件

4.1.2. 安装 MCUXpresso Installer

访问 <https://www.nxp.com/design/design-center/software/development-software/mcuxpresso-software-and-tools/mcuxpresso-installer:MCUXPRESSO-INSTALLER>，进入 MCUXpresso Installer 下载页面。选择适用于 Windows 系统的安装包 MCUXpresso Installer (Windows)，然后下载并安装，如下图所示。

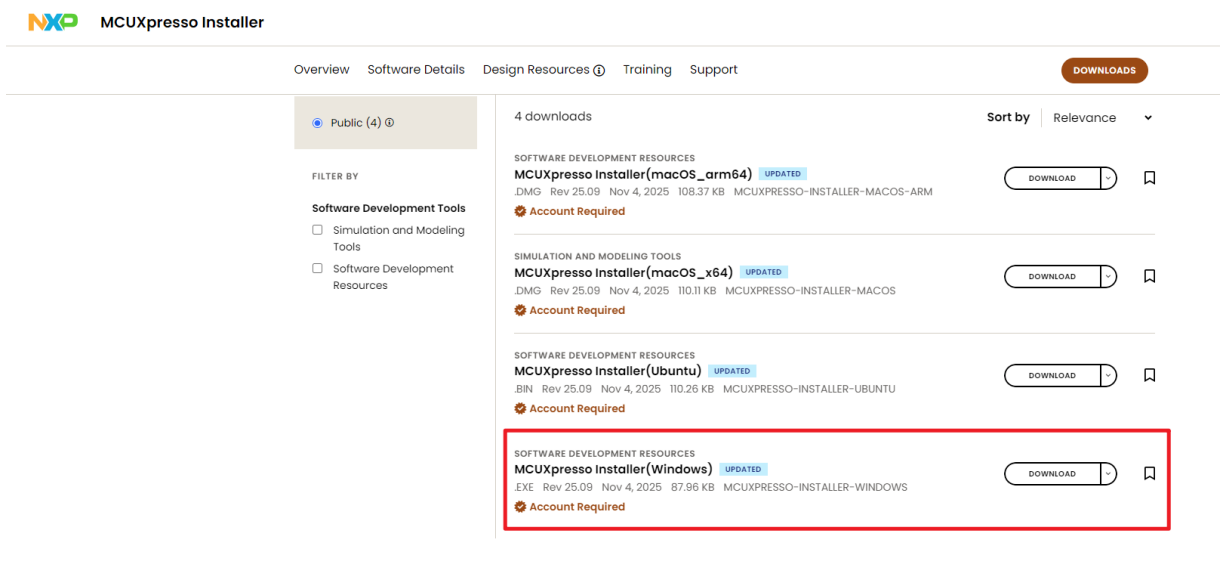


图 6: MCUXpresso Installer 下载页面

4.1.3. 安装 Zephyr 工具链和安装程序

使用 MCUXpresso Installer 工具安装 SDK、工具链并配置开发环境。安装步骤如下。

- 步骤一： 安装 MCUXpresso Installer 后，找到 MCUXpresso Installer 应用程序并启动。
- 步骤二： 在 MCUXpresso Installer 操作界面中，勾选“**Zephyr Developer**”和“**MCUXpresso SDK Developer**”选项，然后单击界面右上角的“**Install**”按钮开始安装。安装成功后，对应选项旁将显示绿色对钩。

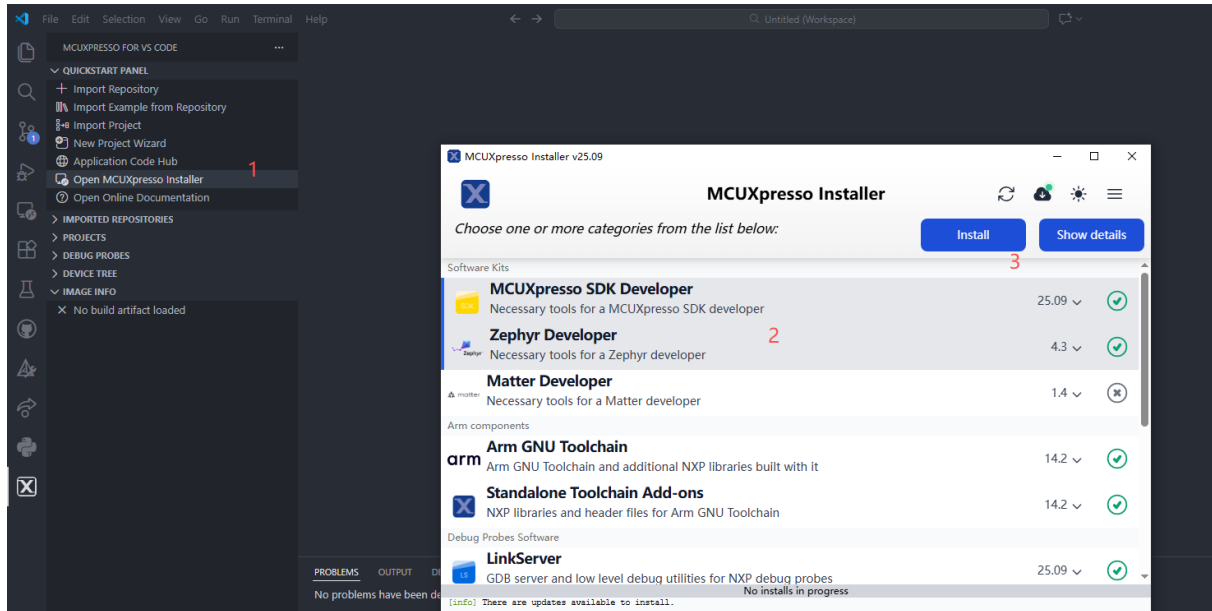


图 7：MCUXpresso Installer 界面

4.1.4. 安装 Zephyr 代码仓库

使用 MCUXpresso Installer 工具克隆并配置 Zephyr 代码仓库，具体步骤如下：

- 步骤一： 单击页面左侧的“**Import Repository**”选项，打开配置窗口。在“**Repository**”下拉菜单中选择“**Zephyr**”（系统会自动填入默认地址：<https://github.com/zephyrproject-rtos/zephyr>）；在“**Revision**”下拉菜单中选择所需版本（例如 v4.3.0）；在“**Location**”字段中单击“**Browse...**”按钮，选择存放目录。
- 步骤二： 单击窗口底部的“**Import**”按钮，克隆 Zephyr 基线代码并将其下载到“**Location**”字段指定的目录中，如下图所示。

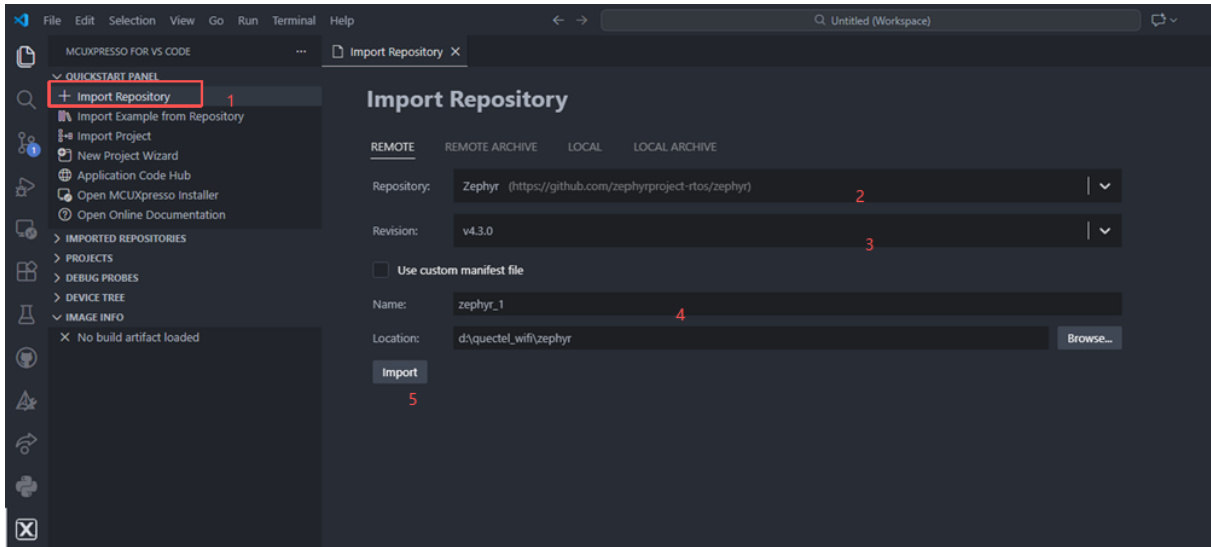


图 8：导入 SDK

4.1.5. 安装 MCUXpresso 工具

参考第 4.1.2 章安装如下 MCUXpresso 工具。

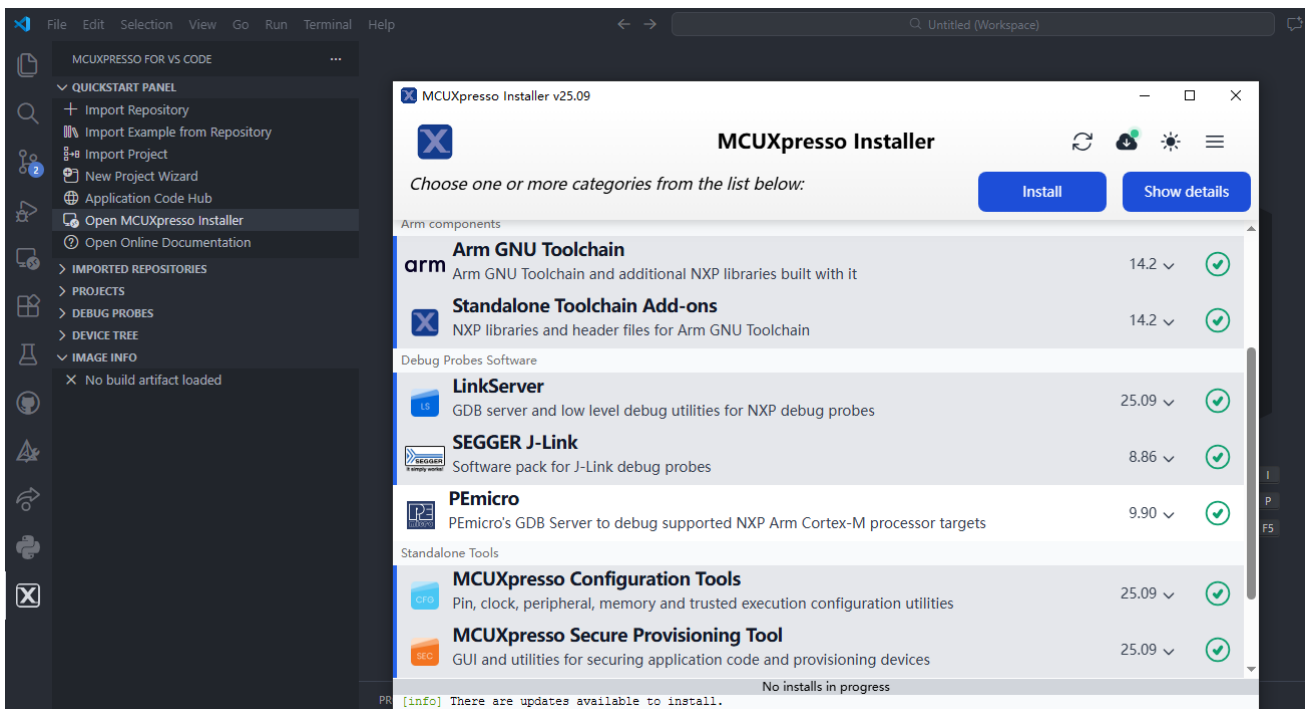


图 9：安装 MCUpresso 工具

4.1.6. 安装问题及解决方法

本章介绍安装过程中可能遇到的问题及相应的解决方法。

1) 问题 1：无法执行 **west --version** 命令。

```
5-12-16T20:36:11.054] [proc] [info] Cmd: where python
[2025-12-16T20:36:11.273] [proc] [info] Cmd: C:\Program Files\Python37\python.exe --version
[2025-12-16T20:36:11.334] [proc] [info] Cmd: ""C:\Program Files\Python37\python.exe"" -m west --version
[2025-12-16T20:36:11.435] [proc] [error] "C:\Program Files\Python37\python.exe" exited with code 1
```

解决方法：升级 pip，并在 Python 3.10 环境中安装 west 工具。

```
C:\Users\username\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip
C:\Users\username\AppData\Local\Programs\Python\Python310\python.exe -m pip install west
```

2) 问题 2：找不到 Python。

```
[2025-12-16T20:46:02.498] [error] Could not find Python. Please make sure it is installed and try again.
```

解决方法：从 Python 官方网站下载并安装适用于 Windows 的 Python 3:

Install python3: <https://www.python.org/downloads/windows/>

3) 问题 3：未找到名为 requests 的模块。

```
D:\quectel_wifi\zephyr\zephyr\modules\hal\nxp\zephyr\blobs\libcsi\cm33\libcsi.a
The module for fetcher "http" could not be imported (No module named 'requests'). This most likely means it is not handling its dependencies properly. Please report this to the zephyr developers.
```

解决方法：执行如下命令，在目标 Python 3.10 环境中安装 requests 库。该环境应与安装 west 的 Python 环境保持一致。

```
C:\Users\username\AppData\Local\Programs\Python\Python310\python.exe -m pip install requests
```

4) **Question 4:**未找到 7-Zip 或 7-Zip 不可用。

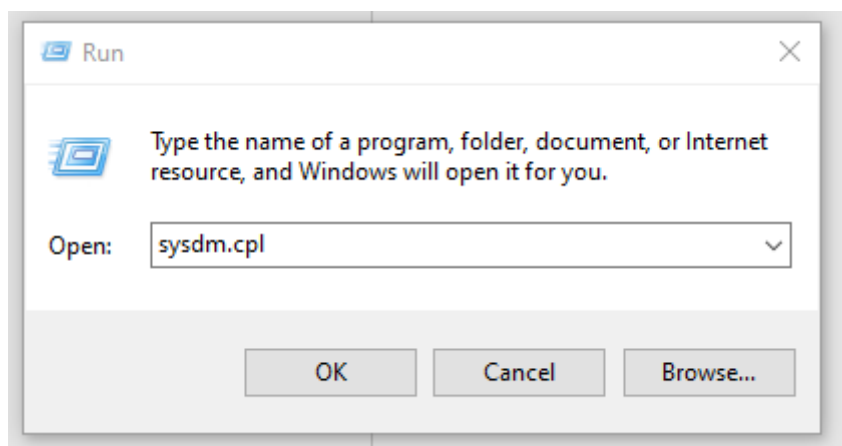


图 10: 打开 sysdm.cpl

解决方法: 将 7-Zip 的安装路径添加到环境变量中, 如下所示:

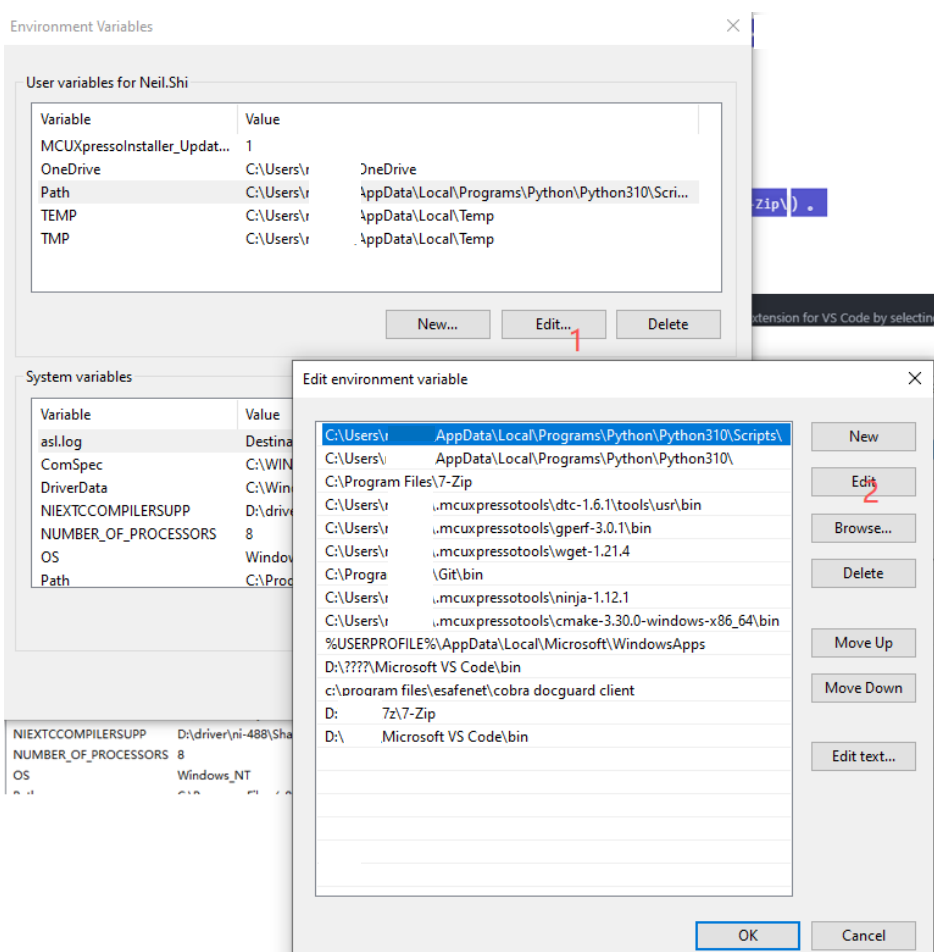


图 11: 添加变量

5) 问题 5: 无法启动 **arm-zephyr-eabi-gcc.exe**。

解决方法: 进入 Zephyr 安装目录并安装所需的依赖项。

```
cd D:\quectel_wifi\zephyr\zephyr
west zephyr-export
pip install -r zephyr/scripts/requirements.txt
```

4.2. 在 VS Code 中编译 M33 镜像

本章介绍如何使用 Zephyr 源码中的 `hello_world` 例程编译 M33 镜像。

4.2.1. 配置 Zephyr 工程

按照下图所示配置相关信息。配置完成后，单击“**Import**”按钮，导入 `hello_world` 例程代码。

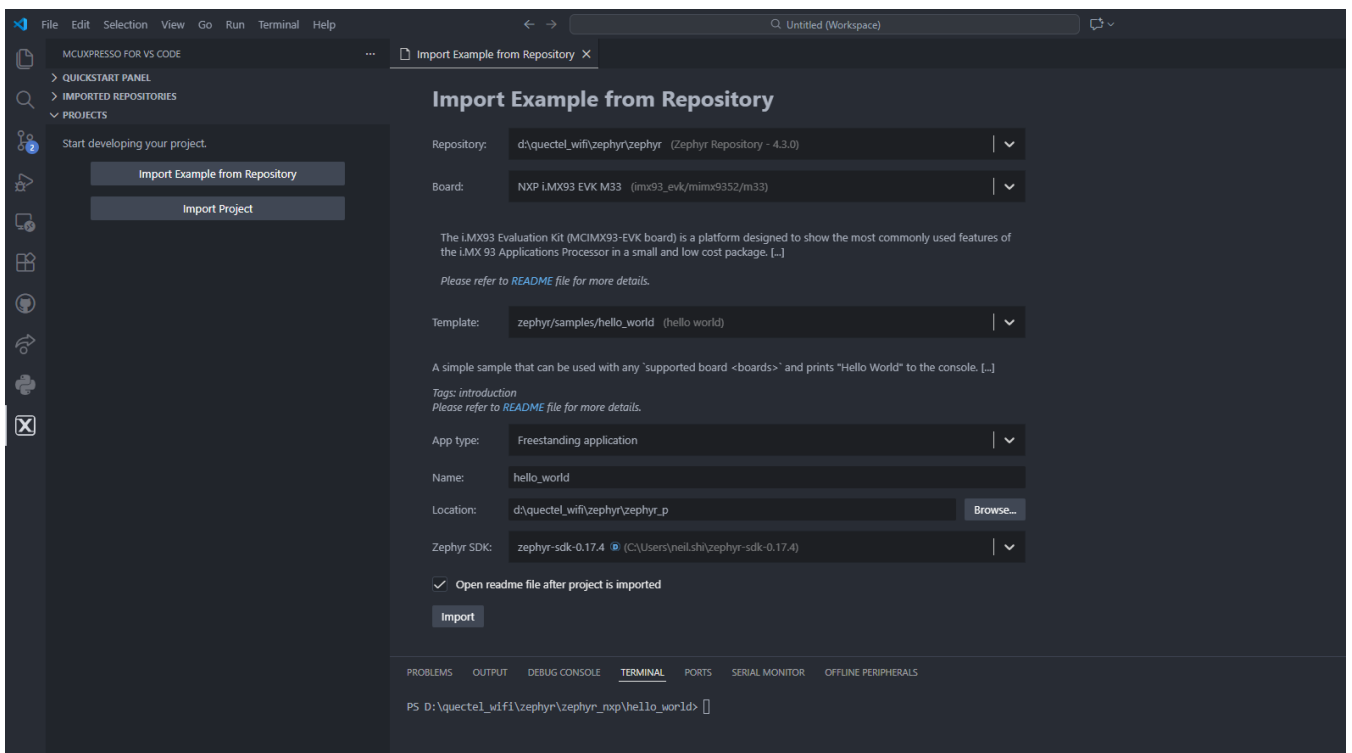


图 12: 导入 `hello_world` 例程代码

导入完成后，界面如下图所示：

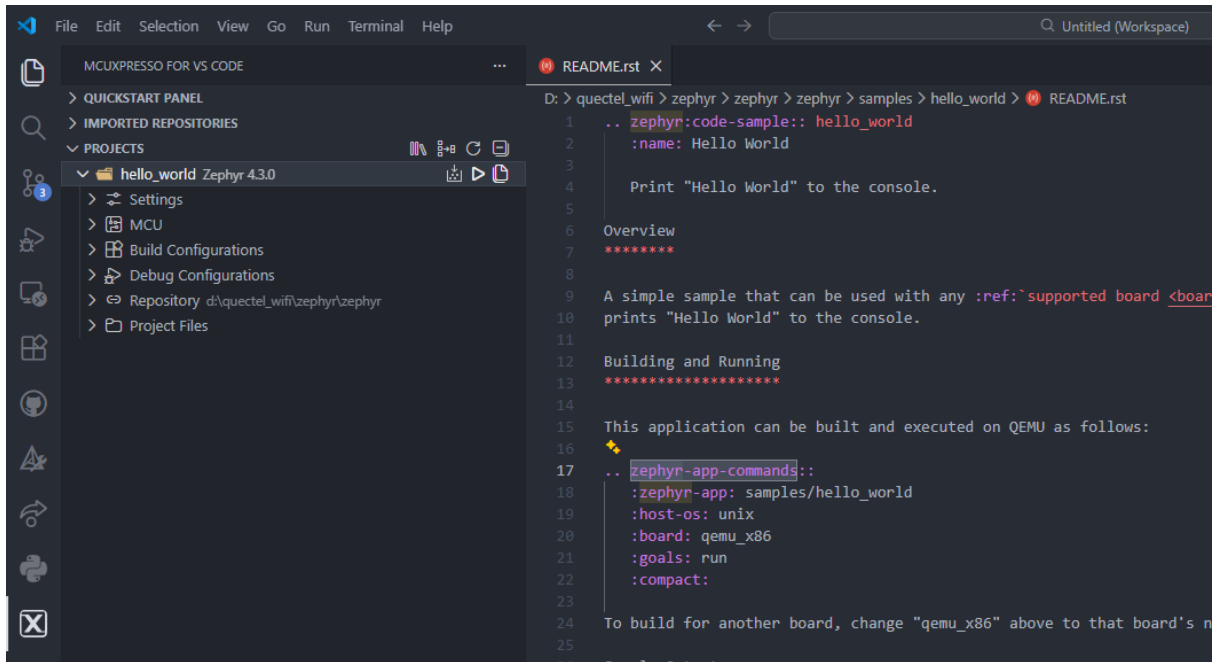


图 13: 导入成功后的界面

导入的 Zephyr 工程中的关键文件如下所示：

表 5: DTS 文件

层级	路径
SoC 基础 DTS（芯片级，不建议修改）	<code><zephyr_install_dir>\zephyr\dts\arm\armv8-m.dtsi</code> <code><zephyr_install_dir>\zephyr\dts\arm\nxp\nxp_imx93_m33.dtsi</code> <code><zephyr_install_dir>\zephyr\modules\hal\nxp\dts\nxp_imx9352cvuxk-pinctrl.dtsi</code>
板级自定义 DTS（开发板级，主要修改位置）	<code><zephyr_install_dir>\zephyr\boards\nxp\imx93_evk\imx93_evk-pinctrl.dtsi</code> <code><zephyr_install_dir>\zephyr\boards\nxp\imx93_evk\imx93_evk_mimx9352_m33.dts</code>

表 6: 内核配置文件

配置类型	路径
默认配置	<code><zephyr_install_dir>/zephyr/boards/nxp/imx93_evk/imx93_evk_mimx9352_m33_defconfig</code>

自定义配置

<zephyr_install_dir>/zephyr/zephyr_p/hello_world/prj.conf

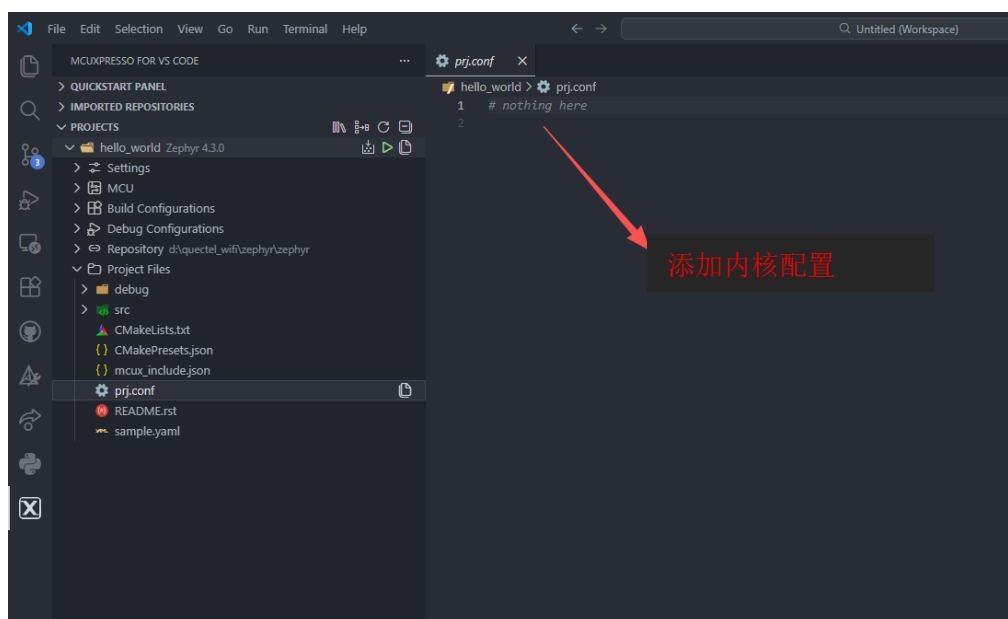


图 14：配置内核

4.2.2. 编译 M33 镜像

单击下图所示的“**Build**”按钮开始编译。

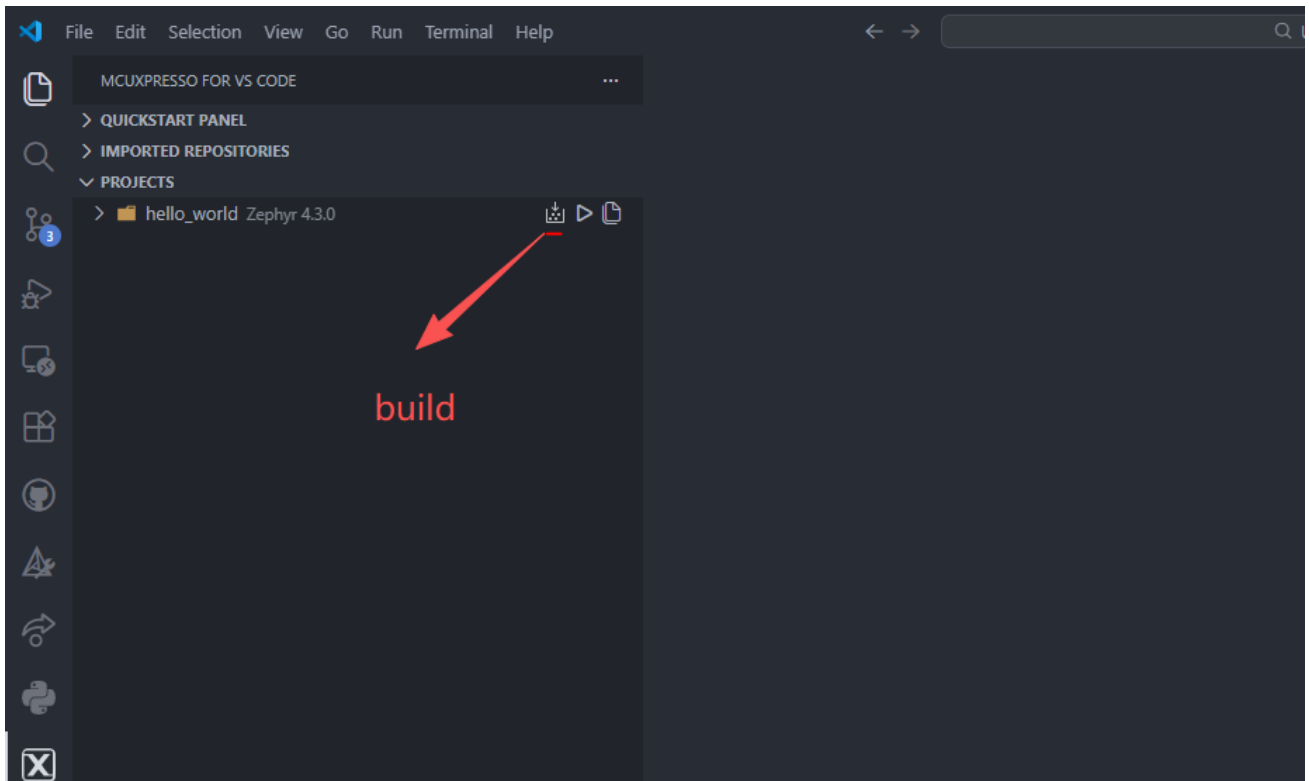


图 15: 编译镜像

M33 镜像编译成功后，界面如下图所示。

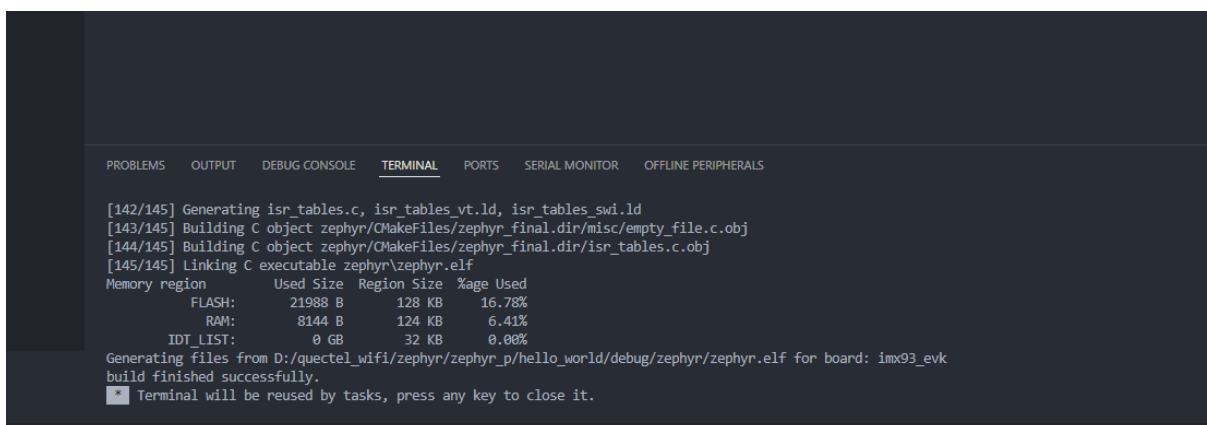


图 16: 编译成功

编译成功后，将在 `zephyr` 目录下的 `build` 目录中生成以下文件：

- `zephyr.bin/zephyr.elf`: 可供 M33 使用的固件镜像文件。
- `.dts` 文件：编译时使用的完整 DTS 文件

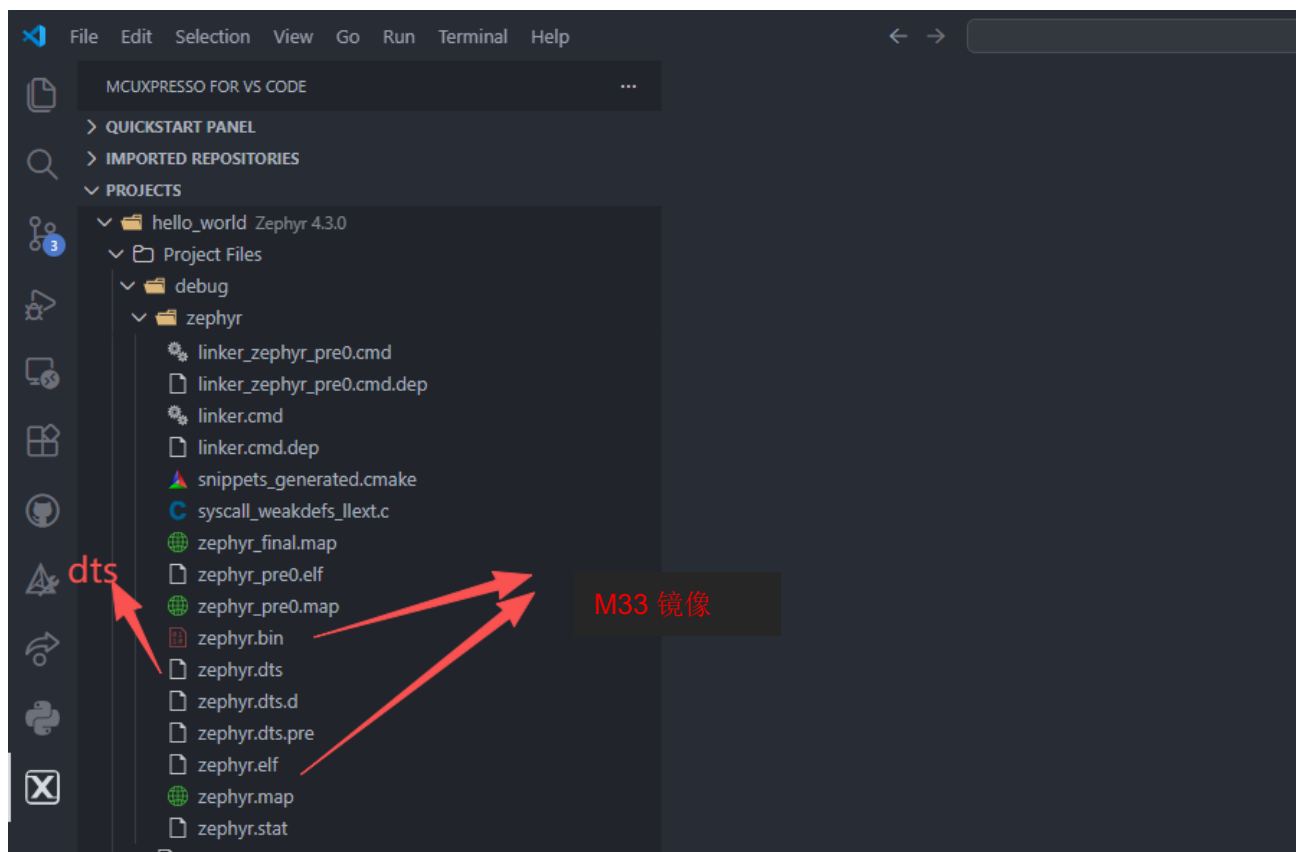


图 17：编译生成的文件

4.3. 在 Ubuntu 中编译 M33 镜像

在 Ubuntu 中执行如下命令编译 M33 镜像。如需了解更多详细信息，也可参考：
https://docs.zephyrproject.org/latest/develop/getting_started/index.html。

```
west build -p always -b frdm_imx93/mimx9352/m33 samples/hello_world
```

4.4. 集成 M33 镜像

4.4.1. 复制 M33 镜像至 Boot 分区

按照第 4.2.2 章和第 4.3 章的说明，M33 的 Zephyr 镜像已编译完成。

可通过多种方式将 *Zephyr.bin* 文件复制到 Boot 分区。本章以 SD 卡启动方式为例，执行如下命令，手动将 M33 镜像复制到 Boot 分区。

```
mount /dev/mmcblk1p1 /mnt
cp Zephyr.bin /mnt/mcore-demos
```

备注

建议使用 **scp** 命令将 *zephyr.bin* 文件直接复制到 */mnt/mcore-demos* 目录。

4.4.2. 打包 M33 镜像到 Rootfs 分区

步骤一：执行如下命令加载编译环境。

```
MACHINE=imx93quec DISTRO=fsl-imx-xwayland source imx-setup-release.sh -b build-
imx93quec
```

```
neil@iyuan-PowerEdge-R740xd:~/SR-IMX93/yocto$ MACHINE=imx93quec DISTRO=fsl-imx-xwayland source imx-setup-release.sh -b build-imx93quec
neil@iyuan-PowerEdge-R740xd:~/SR-IMX93/yocto$ MACHINE=imx93quec DISTRO=fsl-imx-xwayland source imx-setup-release.sh -b build-imx93quec

Build directory is build-imx93quec

Welcome to Freescale Community BSP

The Yocto Project has extensive documentation about OE including a
reference manual which can be found at:
  http://yoctoproject.org/documentation

For more information about OpenEmbedded see their website:
  http://www.openembedded.org/

You can now run 'bitbake <target>'

Common targets are:
  core-image-minimal
  meta-toolchain
  meta-toolchain-sdk
  adt-installer
  meta-ide-support

Your configuration files at build-imx93quec have not been touched.
BSPDIR=
BUILD_DIR=.
neil@iyuan-PowerEdge-R740xd:~/SR-IMX93/yocto/build-imx93quec$
```

步骤二：执行如下命令将 ZST 镜像解压为 WIC 镜像。

```
cd build-imx93quec/tmp/deploy/images/imx93quec
zstd -d imx-image-multimedia-imx93quec.rootfs.wic.zst -o rootfs.wic
```

步骤三：执行如下命令复制 `zephyr.bin` 至 WIC 镜像包。

```
wic cp zephyr.bin rootfs.wic:1/mcore-demos
wic ls rootfs.wic:1/mcore-demos
```

```
neil@yiyuan-PowerEdge-R740xd:~/SR-IMX93/yocto/build-imx93quec/tmp/deploy/images/imx93quec$ wic cp zephyr.bin rootfs.wic:1/mcore-demos
neil@yiyuan-PowerEdge-R740xd:~/SR-IMX93/yocto/build-imx93quec/tmp/deploy/images/imx93quec$ wic ls rootfs.wic:1/mcore-demos
Volume in drive : is boot
Volume Serial Number is 2C47-945D
Directory for ::/mcore-demos
.          <DIR>      2011-04-05  23:00
..         <DIR>      2011-04-05  23:00
IMX93-~1  BIN      47472 2011-04-05  23:00  imx93-11x11-evk_m33_TCM_sai_low_power_audio.bin
IMX93-~2  BIN      19080 2011-04-05  23:00  imx93-11x11-evk_m33_TCM_power_mode_switch.bin
IMX93-~3  BIN      130428 2011-04-05  23:00  imx93-11x11-evk_m33_TCM_low_power_wakeword.bin
IMX93-~4  BIN      18708 2011-04-05  23:00  imx93-11x11-evk_m33_TCM_rpmsg_lite_pingpong_rtos_linux_remote.bin
RPMMSG_~1 BIN      39092 2011-04-05  23:00  rpmsg_lite_str_echo_rtos_remote.bin
IMX93-~5  BIN      17228 2011-04-05  23:00  imx93-11x11-evk_m33_TCM_rpmsg_lite_str_echo_rtos.bin
zephyr   bin       21988 2025-12-22  17:25
          9 files          293 996 bytes
          309 944 320 bytes free
```

步骤四：执行如下命令打包镜像为 ZST 格式。

```
zstd rootfs.wic
rootfs.wic.zst
```

```
309 944 320 bytes free
@yiyuan-PowerEdge-R740xd:~/SR-IMX93/yocto/build-imx93quec/tmp/deploy/images/imx93quec$ zstd rootfs.wic
zstd: rootfs.wic.zst already exists; overwrite (y/n) ? y
rootfs.wic      : 13.12% ( 3.85 GiB => 517 MiB, rootfs.wic.zst)
```

步骤五：参考文档 [1]将镜像烧录至模块中。

4.5. 启动验证

执行以下命令启动 Zephyr 镜像。有关详细信息，请参见第 2.3 章。

```
u-boot => fatload mmc 0:1 80000000 mcore-demos/zephyr.bin
u-boot => cp.b 0x80000000 0x201e0000 0x10000
u-boot => bootaux 0x1ffe0000 0
```

如下图所示，在 A55 核的 U-Boot 阶段，将 M33 核的固件复制到 TCM，然后启动 M33 核。如果显示与下图类似的信息，则表示 M33 已成功启动。

A55 核输出:

```
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=>
u-boot=> fatload mmc 0:1 80000000 mcore-demos/zephyr.bin
21988 bytes read in 8 ms (2.6 MiB/s)
u-boot=> cp.b 0x80000000 0x201e0000 0x10000
u-boot=> bootaux 0x1ffe0000 0
## Starting auxiliary core addr = 0x1FFE0000...
u-boot=>
```

M33 核输出:

```
*** Booting Zephyr OS build v4.3.0 ***
Hello World! imx93_evk/mimx9352/m33
```

5 A55 核的 Zephyr 开发

虽然本文主要介绍 M33 核，但在介绍 Zephyr 时，也有必要说明如何在 A55 核上运行 Zephyr。可参考第 4 章搭建 Zephyr 开发环境，然后执行以下构建命令，生成适用于 A55 核的 Zephyr 镜像。

步骤一：在 `zephyrproject\zephyr\samples\hello_world\prj.conf` 文件中添加以下配置，以支持 1 GB DDR。

```
CONFIG_SRAM_BASE_ADDRESS=0x80000000
CONFIG_KERNEL_VM_BASE=0x80000000
```

步骤二：在 Ubuntu 中执行如下命令，为 A55 核编译 Zephyr 镜像。

```
west build -p always -b frdm_imx93/mimx9352/a55 samples/hello_world
```

步骤三：执行如下命令，手动将 `zephyr.bin` 文件复制到 Boot 分区。

```
mount /dev/mmcblk1p1 /mnt
cp Zephyr.bin /mnt
```

备注

建议使用 `scp` 命令将 `zephyr.bin` 文件直接复制到 `/mnt` 目录。

步骤四：执行如下命令运行 Zephyr。

```
u-boot=> fatload mmc 0:1 0x80000000 zephyr.bin
u-boot=> dcache flush
u-boot=> icache flush
u-boot=> go 0x80000000
```

如果显示以下日志，则表示 Zephyr 已在 A55 核上运行。

```
Hello World! frdm_imx93/mimx9352/a55
```

6 附录 参考文档及术语缩写

表 7：参考文档

文档名称
[1] Quectel_SRG091X&SRG093X 系列_编译&烧录指导

表 8：术语缩写

缩写	英文全称	中文全称
API	Application Programming Interface	应用程序编程接口
DDR	Double Data Rate	双倍数据速率
DTB	Device Tree Blob	设备树二进制文件
DTS	Device Tree Source	设备树源文件
eMMC	Embedded MultiMedia Card	嵌入式多媒体卡
ECC	Error Correction Code	错误校正码
EVK	Evaluation Kit	评估套件
FPU	Floating Point Unit	浮点运算单元
I2C	Inter-Integrated Circuit	集成电路总线
IDE	Integrated Development Environment	集成开发环境
LwIP	Lightweight IP	轻量级 IP 协议栈
MCU	Microcontroller Unit	微型控制单元
MPU	Memory Protection Unit	内存保护单元
NVIC	Nested Vectored Interrupt Controller	嵌套向量中断控制器

OCRAM	On-Chip Random Access Memory	片上随机存取存储器
PMIC	Power Management Integrated Circuit	电源管理集成电路
RTOS	Real-Time Operating System	实时操作系统
SoC	System on Chip	片上系统
SD	Secure Digital	安全数字存储卡
SDK	Software Development Kit	软件开发套件
SPI	Serial Peripheral Interface	串行外设接口
TCM	Tightly Coupled Memory	紧耦合存储器
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器
USB	Universal Serial Bus	通用串行总线